

Лабораторная работа № 3.1

«Самоприменимый генератор компиляторов на основе предсказывающего анализа»

18 мая 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является изучение алгоритма построения таблиц предсказывающего анализатора.

Индивидуальный вариант

– аксиома заключена

– в фигурные скобки

$T, T', \{ E \}, E', F$

$\langle E : T E' \rangle$

$\langle E' : "+" T E' : @ \rangle$

$\langle T : F T' \rangle$

$\langle T' : "*" F T' : @ \rangle$

$\langle F : "n" : "(" E ")" \rangle$

Грамматика на входном языке

{ Program }, DeclList, DeclListTail, Decl, Rules,

Rule, Alts, AltsTail, Alt, Seq, SeqTail, Symbol

$\langle \text{Program} : \text{DeclList Rules} \rangle$

$\langle \text{DeclList} : \text{Decl DeclListTail} \rangle$

$\langle \text{DeclListTail} : \text{"COMMA"} \text{Decl DeclListTail} : @ \rangle$

```

< Decl : "NONTERM" : "LBRACE" "NONTERM" "RBRACE" >
< Rules : Rule Rules : @ >
< Rule : "LANGLE" "NONTERM" "COLON" Alts "RANGLE" >
< Alts : Alt AltsTail >
< AltsTail : "COLON" Alt AltsTail : @ >
< Alt : "EPS" : Seq >
< Seq : Symbol SeqTail >
< SeqTail : Symbol SeqTail : @ >
< Symbol : "NONTERM" : "STRING" >

```

Реализация

Генератор компиляторов

```

from __future__ import annotations

import argparse
from pathlib import Path
from typing import Any, Dict, List, Optional, Set, Tuple

from dataclasses import dataclass, field

from self_generated_table import SPEC_TABLE

EPSILON = "@"
EOF_SYMBOL = "$"
...

SPEC_TABLE: Dict[str, Dict[str, List[str]]] = {
    "Program": {
        "NONTERM": ["DeclList", "Rules"],
        "LBRACE": ["DeclList", "Rules"],
    },
    "DeclList": {
        "NONTERM": ["Decl", "DeclListTail"],
        "LBRACE": ["Decl", "DeclListTail"],
    },
    "DeclListTail": {
        "COMMA": ["COMMA", "Decl", "DeclListTail"],
        "LANGLE": [],
        EOF_SYMBOL: [],
    },
    "Decl": {
        "NONTERM": ["NONTERM"],
        "LBRACE": ["LBRACE", "NONTERM", "RBRACE"],
    },
}

```

```

"Rules": {
    "LANGLE": ["Rule", "Rules"],
    EOF_SYMBOL: [],
},
"Rule": {
    "LANGLE": ["LANGLE", "NONTERM", "COLON", "Alts", "RANGLE"],
},
"Alts": {
    "EPS": ["Alt", "AltsTail"],
    "NONTERM": ["Alt", "AltsTail"],
    "STRING": ["Alt", "AltsTail"],
},
"AltsTail": {
    "COLON": ["COLON", "Alt", "AltsTail"],
    "RANGLE": [],
},
"Alt": {
    "EPS": ["EPS"],
    "NONTERM": ["Seq"],
    "STRING": ["Seq"],
},
"Seq": {
    "NONTERM": ["Symbol", "SeqTail"],
    "STRING": ["Symbol", "SeqTail"],
},
"SeqTail": {
    "NONTERM": ["Symbol", "SeqTail"],
    "STRING": ["Symbol", "SeqTail"],
    "COLON": [],
    "RANGLE": [],
},
"Symbol": {
    "NONTERM": ["NONTERM"],
    "STRING": ["STRING"],
},
}
'''
class LexError(Exception):
    pass

class ParseError(Exception):
    pass

class GrammarError(Exception):

```

```

pass

@dataclass(frozen=True)
class Token:
    type: str
    lexeme: str
    line: int
    col: int

@dataclass
class Node:
    symbol: str
    token: Optional[Token] = None
    children: List["Node"] = field(default_factory=list)

@dataclass(frozen=True)
class SymbolRef:
    kind: str
    name: str
    line: int
    col: int

@dataclass(frozen=True)
class AlternativeSpec:
    symbols: List[SymbolRef]
    line: int
    col: int

@dataclass(frozen=True)
class RuleSpec:
    lhs: SymbolRef
    alternatives: List[AlternativeSpec]

@dataclass
class GrammarSpec:
    axioms: List[SymbolRef]
    declarations: List[SymbolRef]
    rules: List[RuleSpec]

```

```

@dataclass(frozen=True)
class Production:
    lhs: str
    rhs: Tuple[str, ...]
    line: int
    col: int

class GrammarSpecLexer:
    def __init__(self, src: str):
        self.src = src
        self.i = 0
        self.line = 1
        self.col = 1

    def peek(self, k: int = 0) -> str:
        idx = self.i + k
        return self.src[idx] if idx < len(self.src) else "\0"

    def advance(self) -> str:
        ch = self.peek()
        if ch == "\0":
            return ch
        self.i += 1
        if ch == "\n":
            self.line += 1
            self.col = 1
        else:
            self.col += 1
        return ch

    def next_token(self) -> Token:
        while True:
            ch = self.peek()
            if ch == "\0":
                return Token(EOF_SYMBOL, "", self.line, self.col)
            if ch.isspace():
                self.advance()
                continue
            if ch == "-" and self.peek(1) == "-":
                while self.peek() not in ("\0", "\n"):
                    self.advance()
                continue
            break

        line, col = self.line, self.col
        ch = self.peek()

```

```

singles = {
    "<": "LANGLE",
    ">": "RANGLE",
    "{": "LBRACE",
    "}": "RBRACE",
    ":": "COLON",
    ",": "COMMA",
    "@": "EPS",
}
if ch in singles:
    self.advance()
    return Token(singles[ch], ch, line, col)

if ch == "'":
    self.advance()
    buf: List[str] = []
    while True:
        cur = self.peek()
        if cur == "\\0" or cur == "\\n":
            raise LexError(f"Unterminated \
                           string literal at {line}:{col}")
        if cur == "'":
            self.advance()
            break
        if cur == "\\":
            self.advance()
            esc = self.peek()
            if esc == "\\0" or esc == "\\n":
                raise LexError(f"Unterminated \
                               escape sequence at {line}:{col}")
            mapping = {'\\': '\\', "\\\\": "\\\\", \
                      "n": "\\n", "t": "\\t", "r": "\\r"}
            buf.append(mapping.get(esc, esc))
            self.advance()
        else:
            buf.append(self.advance())
    return Token("STRING", "".join(buf), line, col)

if "A" <= ch <= "Z":
    buf = [self.advance()]
    while True:
        cur = self.peek()
        if cur.isalnum() or cur == "_":
            buf.append(self.advance())
        elif cur == "'":

```

```

        buf.append(self.advance())
    else:
        break
    return Token("NONTERM", "".join(buf), line, col)

    raise LexError(f"Unexpected character '{ch}' at {line}:{col}")

@dataclass
class PreparedGrammar:
    start_symbol: str
    nonterminals: Set[str]
    terminals: Set[str]
    productions_by_lhs: Dict[str, List[Production]]

class LL1Analyzer:
    def __init__(self, grammar: GrammarSpec):
        self.grammar = grammar
        self.declared_order = [ref.name for ref in grammar.declarations]
        self.declared_map = {ref.name: ref for ref in grammar.declarations}
        self.axiom = grammar.axioms[0].name if grammar.axioms else None
        self.prepared: Optional[PreparedGrammar] = None

    def prepare(self) -> PreparedGrammar:
        seen: Dict[str, SymbolRef] = {}
        for ref in self.grammar.declarations:
            if ref.name in seen:
                raise GrammarError(f"Grammar error at \{ref.line}\
                                   :{ref.col}: symbol '{ref.name}' declared twice")
            seen[ref.name] = ref

        if not self.grammar.axioms:
            tok = self.grammar.declarations[0] if self.grammar.declarations else None
            if tok is None:
                raise GrammarError("Grammar error: no axiom declared")
            raise GrammarError(f"Grammar error at {tok.line}\
                               :{tok.col}: no axiom declared")

        if len(self.grammar.axioms) > 1:
            ref = self.grammar.axioms[1]
            raise GrammarError(f"Grammar error at {ref.line}:{ref.col}:\
                               more than one axiom declared")

        nonterminals = set(seen)
        terminals: Set[str] = set()
        productions_by_lhs: Dict[str, List[Production]] = {nt: [] for nt in nonterminals}

```

```

if self.axiom not in nonterminals:
    ax = self.grammar.axioms[0]
    raise GrammarError(f"Grammar error at {ax.line}:{ax.col}:\
                        axiom '{ax.name}' is not declared")

for rule in self.grammar.rules:
    if rule.lhs.kind != "NONTERM" or rule.lhs.name not in nonterminals:
        raise GrammarError(f"Grammar error at {rule.lhs.line}:{rule.lhs.col}:\
                            undeclared nonterminal '{rule.lhs.name}'")
    for alt in rule.alternatives:
        rhs: List[str] = []
        for sym in alt.symbols:
            if sym.kind == "NONTERM":
                if sym.name not in nonterminals:
                    raise GrammarError(f"Grammar error at {sym.line}:{sym.col}:\
                                        undeclared nonterminal '{sym.name}'")
                rhs.append(sym.name)
            else:
                terminals.add(sym.name)
                rhs.append(sym.name)
        productions_by_lhs[rule.lhs.name].append(\
            Production(rule.lhs.name, tuple(rhs), alt.line, alt.col))

lhs_set = {prod.lhs for prods in productions_by_lhs.values() for prod in prods}
for nt, ref in self.declared_map.items():
    if nt not in lhs_set:
        raise GrammarError(
            f"Grammar error at {ref.line}:{ref.col}: \
            nonterminal '{nt}' does not appear on the left-
hand side of any rule"
        )

self.prepared = PreparedGrammar(
    start_symbol=self.axiom,
    nonterminals=nonterminals,
    terminals=terminals,
    productions_by_lhs=productions_by_lhs,
)
return self.prepared

def analyze(self) -> \
    Tuple[PreparedGrammar, Dict[str, List[Production]], \
        Dict[str, Set[str]], Dict[str, Set[str]]]:

    prepared = self.prepare()
    first, nullable = self.compute_first(prepared)

```

```

        follow = self.compute_follow(prepared, first, nullable)
        self.check_ll1(prepared, first, nullable, follow)
        return prepared, prepared.productions_by_lhs, first, follow

def compute_first(self, g: PreparedGrammar) -> \
    Tuple[Dict[str, Set[str]], Dict[str, bool]]:

    first: Dict[str, Set[str]] = {nt: set() for nt in g.nonterminals}
    nullable: Dict[str, bool] = {nt: False for nt in g.nonterminals}

    changed = True
    while changed:
        changed = False
        for lhs, prods in g.productions_by_lhs.items():
            for prod in prods:
                rhs = prod.rhs
                if not rhs:
                    if not nullable[lhs]:
                        nullable[lhs] = True
                        changed = True
                    continue
                all_nullable = True
                for sym in rhs:
                    if sym in g.nonterminals:
                        before = len(first[lhs])
                        first[lhs].update(first[sym])
                        if len(first[lhs]) != before:
                            changed = True
                        if not nullable[sym]:
                            all_nullable = False
                            break
                    else:
                        if sym not in first[lhs]:
                            first[lhs].add(sym)
                            changed = True
                        all_nullable = False
                        break
                if all_nullable and not nullable[lhs]:
                    nullable[lhs] = True
                    changed = True

    return first, nullable

def first_of_sequence(self, seq: Tuple[str, ...], \
    first: Dict[str, Set[str]], \
    nullable: Dict[str, bool], \

```

```

        nonterminals: Set[str]) -> Set[str]:
    out: Set[str] = set()
    if not seq:
        out.add(EPSILON)
        return out
    for sym in seq:
        if sym in nonterminals:
            out.update(first[sym])
            if not nullable[sym]:
                break
        else:
            out.add(sym)
            break
    else:
        out.add(EPSILON)
    return out

def compute_follow(self, g: PreparedGrammar, \
    first: Dict[str, Set[str]], \
    nullable: Dict[str, bool]) -> Dict[str, Set[str]]:
    follow: Dict[str, Set[str]] = {nt: set() for nt in g.nonterminals}
    follow[g.start_symbol].add(EOF_SYMBOL)

    changed = True
    while changed:
        changed = False
        for lhs, prods in g productions_by_lhs.items():
            for prod in prods:
                rhs = prod.rhs
                for i, sym in enumerate(rhs):
                    if sym not in g.nonterminals:
                        continue
                    beta = rhs[i + 1 :]
                    first_beta = self.first_of_sequence(beta, \
                        first, nullable, g.nonterminals)
                    before = len(follow[sym])
                    follow[sym].update(x for x in first_beta if x != EPSILON)
                    if EPSILON in first_beta or not beta:
                        follow[sym].update(follow[lhs])
                    if len(follow[sym]) != before:
                        changed = True

    return follow

def predict_set(self, \
    prod: Production, \
    first: Dict[str, Set[str]], \

```

```

        nullable: Dict[str, bool], \
        follow: Dict[str, Set[str]], \
        nonterminals: Set[str]) -> Set[str]:
seq_first = self.first_of_sequence(prod.rhs, first, nullable, nonterminals)
if EPSILON in seq_first:
    return (seq_first - {EPSILON}) | follow[prod.lhs]
return seq_first

def check_ll1(self, g: PreparedGrammar, \
    first: Dict[str, Set[str]], \
    nullable: Dict[str, bool], \
    follow: Dict[str, Set[str]]) -> None:
for lhs, prods in g productions_by_lhs.items():
    seen: Dict[str, Production] = {}
    for prod in prods:
        pred = self.predict_set(prod, first, nullable, follow, g.nonterminals)
        for terminal in pred:
            if terminal in seen:
                other = seen[terminal]
                raise GrammarError(
                    f"Grammar error at {prod.line}:{prod.col}: grammar is \
                        not LL(1); conflict for nonterminal \
                        '{lhs}' on lookahead '{terminal}'"
                )
            seen[terminal] = prod

def build_table(self) -> Tuple[PreparedGrammar, Dict[str, Dict[str, List[str]]]]:
    prepared = self.prepare()
    first, nullable = self.compute_first(prepared)
    follow = self.compute_follow(prepared, first, nullable)
    self.check_ll1(prepared, first, nullable, follow)
    table: Dict[str, Dict[str, List[str]]] = {nt: {} for nt in prepared.nonterminals}
    for lhs, prods in prepared productions_by_lhs.items():
        for prod in prods:
            pred = self.predict_set(prod, first, nullable, follow, prepared.nonterminals)
            for terminal in pred:
                if terminal in table[lhs]:
                    raise GrammarError(f"Grammar error at \{prod.line}:{prod.col}:\
                        grammar is not LL(1); duplicate table \
                        entry for {lhs}, {terminal}")
                    table[lhs][terminal] = list(prod.rhs)
    return prepared, table

class LL1Parser:
    def __init__(self, table: Dict[str, Dict[str, List[str]]], start_symbol: str):
        self.table = table

```

```

self.start_symbol = start_symbol

def parse(self, tokens: List[Token]) -> Node:
    if not tokens or tokens[-1].type != EOF_SYMBOL:
        tokens = list(tokens) + [Token(EOF_SYMBOL, "", tokens[-
1].line \
                                if tokens else 1, tokens[-
1].col if tokens else 1)]
        pos = 0
        root = Node(self.start_symbol)
        stack: List[Tuple[str, Node]] = [(EOF_SYMBOL, Node(EOF_SYMBOL)), \
                                         (self.start_symbol, root)]

    while stack:
        symbol, node = stack.pop()
        tok = tokens[pos]
        if symbol == EOF_SYMBOL:
            if tok.type != EOF_SYMBOL:
                raise ParseError(f"Syntax error at {tok.line}:{tok.col}:\
                                expected EOF, found {tok.type}")
            pos += 1
            continue

        if symbol not in self.table:
            if tok.type != symbol:
                raise ParseError(f"Syntax error at {tok.line}:{tok.col}:\
                                expected '{symbol}', found '{tok.type}'")
            node.token = tok
            pos += 1
            continue

        prod = self.table[symbol].get(tok.type)
        if prod is None:
            expected = ", ".join(sorted(self.table[symbol].keys()))
            raise ParseError(f"Syntax error at {tok.line}:{tok.col}:\
                            expected one of {{{expected}}}, found {tok.type}")
        if prod:
            children = [Node(sym) for sym in prod]
            node.children = children
            for child in reversed(children):
                stack.append((child.symbol, child))
        else:
            node.children = [Node(EPSILON, token=tok)]

    if pos != len(tokens):
        tok = tokens[pos]

```

```

        raise ParseError(f"Syntax error at {tok.line}:{tok.col}:\
                           unexpected trailing input")

    return root

def _token(node: Node) -> Token:
    if node.token is None:
        raise ParseError(f"Internal parse tree error: missing token for '{node.symbol}'")
    return node.token

def grammar_spec_from_tree(root: Node) -> GrammarSpec:
    def parse_decl_list(node: Node) -> Tuple[List[SymbolRef], List[SymbolRef]]:
        if not node.children:
            raise ParseError("Internal parse tree error: empty DeclList")
        decls, axioms = parse_decl(node.children[0])
        parse_decl_list_tail(node.children[1], decls, axioms)
        return decls, axioms

    def parse_decl_list_tail(node: Node, \
                             decls: List[SymbolRef], \
                             axioms: List[SymbolRef]) -> None:
        if len(node.children) == 1 and node.children[0].symbol == EPSILON:
            return
        if len(node.children) != 3:
            raise ParseError("Internal parse tree error: malformed DeclListTail")
        parse_decl(node.children[1], decls, axioms)
        parse_decl_list_tail(node.children[2], decls, axioms)

    def parse_decl(node: Node, \
                   decls: Optional[List[SymbolRef]] = None, \
                   axioms: Optional[List[SymbolRef]] = None):
        if decls is None:
            decls = []
        if axioms is None:
            axioms = []
        if len(node.children) == 1 and node.children[0].symbol == "NONTERM":
            tok = _token(node.children[0])
            decls.append(SymbolRef("NONTERM", tok.lexeme, tok.line, tok.col))
            return decls, axioms
        if len(node.children) == 3 and node.children[0].symbol == "LBRACE":
            tok = _token(node.children[1])
            ref = SymbolRef("NONTERM", tok.lexeme, tok.line, tok.col)
            decls.append(ref)
            axioms.append(ref)
            return decls, axioms
        raise ParseError("Internal parse tree error: malformed Decl")

```

```

def parse_rules(node: Node) -> List[RuleSpec]:
    if len(node.children) == 1 and node.children[0].symbol == EPSILON:
        return []
    if len(node.children) != 2:
        raise ParseError("Internal parse tree error: malformed Rules")
    first = parse_rule(node.children[0])
    rest = parse_rules(node.children[1])
    return [first] + rest

def parse_rule(node: Node) -> RuleSpec:
    if len(node.children) != 5:
        raise ParseError("Internal parse tree error: malformed Rule")
    lhs_tok = _token(node.children[1])
    lhs = SymbolRef("NONTERM", lhs_tok.lexeme, lhs_tok.line, lhs_tok.col)
    alts = parse_alts(node.children[3])
    return RuleSpec(lhs=lhs, alternatives=alts)

def parse_alts(node: Node) -> List[AlternativeSpec]:
    if len(node.children) != 2:
        raise ParseError("Internal parse tree error: malformed Alts")
    first = parse_alt(node.children[0])
    rest = parse_alts_tail(node.children[1])
    return [first] + rest

def parse_alts_tail(node: Node) -> List[AlternativeSpec]:
    if len(node.children) == 1 and node.children[0].symbol == EPSILON:
        return []
    if len(node.children) != 3:
        raise ParseError("Internal parse tree error: malformed AltsTail")
    first = parse_alt(node.children[1])
    rest = parse_alts_tail(node.children[2])
    return [first] + rest

def parse_alt(node: Node) -> AlternativeSpec:
    if len(node.children) == 1 and node.children[0].symbol == "EPS":
        tok = _token(node.children[0])
        return AlternativeSpec(symbols=[], line=tok.line, col=tok.col)
    if len(node.children) != 1:
        raise ParseError("Internal parse tree error: malformed Alt")
    seq = parse_seq(node.children[0])
    return AlternativeSpec(symbols=seq, line=seq[0].line, col=seq[0].col)

def parse_seq(node: Node) -> List[SymbolRef]:
    if len(node.children) != 2:
        raise ParseError("Internal parse tree error: malformed Seq")
    first = parse_symbol(node.children[0])

```

```

        rest = parse_seq_tail(node.children[1])
        return [first] + rest

def parse_seq_tail(node: Node) -> List[SymbolRef]:
    if len(node.children) == 1 and node.children[0].symbol == EPSILON:
        return []
    if len(node.children) != 2:
        raise ParseError("Internal parse tree error: malformed SeqTail")
    first = parse_symbol(node.children[0])
    rest = parse_seq_tail(node.children[1])
    return [first] + rest

def parse_symbol(node: Node) -> SymbolRef:
    if len(node.children) != 1:
        raise ParseError("Internal parse tree error: malformed Symbol")
    tok = _token(node.children[0])
    return SymbolRef(tok.type, tok.lexeme, tok.line, tok.col)

if root.symbol != "Program" or len(root.children) != 2:
    raise ParseError("Internal parse tree error: malformed Program")
declarations, axioms = parse_decl_list(root.children[0])
rules = parse_rules(root.children[1])
return GrammarSpec(axioms=axioms, declarations=declarations, rules=rules)

def read_grammar_spec(src: str) -> GrammarSpec:
    lexer = GrammarSpecLexer(src)
    tokens: List[Token] = []
    while True:
        tok = lexer.next_token()
        tokens.append(tok)
        if tok.type == EOF_SYMBOL:
            break
    parser = LL1Parser(SPEC_TABLE, "Program")
    tree = parser.parse(tokens)
    return grammar_spec_from_tree(tree)

# -----

def py_string(s: str) -> str:
    return repr(s)

def py_list(obj: List[Any], indent: int = 0) -> str:
    if not obj:
        return "[]"
    parts = []

```

```

    for item in obj:
        parts.append(py_expr(item, indent + 4))
    if all("\n" not in p for p in parts) and sum(len(p) for p in parts) < 80:
        return "[" + ", ".join(parts) + "]"
    pad = " " * indent
    inner = ",\n".join(pad + "    " + p for p in parts)
    return "[\n" + inner + "\n" + pad + "]"

def py_dict(obj: Dict[Any, Any], indent: int = 0) -> str:
    if not obj:
        return "{}"
    items = []
    for k, v in obj.items():
        items.append(f"{py_expr(k, indent + 4)}: {py_expr(v, indent + 4)}")
    if all("\n" not in p for p in items) and sum(len(p) for p in items) < 80:
        return "{" + ", ".join(items) + "}"
    pad = " " * indent
    inner = ",\n".join(pad + "    " + p for p in items)
    return "{\n" + inner + "\n" + pad + "}"

def py_expr(obj: Any, indent: int = 0) -> str:
    if isinstance(obj, str):
        return py_string(obj)
    if isinstance(obj, list):
        return py_list(obj, indent)
    if isinstance(obj, dict):
        return py_dict(obj, indent)
    if obj is None:
        return "None"
    if isinstance(obj, bool):
        return "True" if obj else "False"
    return repr(obj)

def write_generated_file(path: Path, start_symbol: str, \
                        table: Dict[str, Dict[str, List[str]]]) -> None:
    content = [
        "from __future__ import annotations",
        "",
        f"START_SYMBOL = {py_string(start_symbol)}",
        f"EOF_SYMBOL = {py_string('$')}",
        f"EPSILON = {py_string('')}",
        f"SPEC_TABLE = {py_expr(table)}",
        "",
    ]

```

```

]
path.write_text("\n".join(content), encoding="utf-8")

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("input")
    ap.add_argument("output")
    args = ap.parse_args()

    src = Path(args.input).read_text(encoding="utf-8")
    try:
        grammar = read_grammar_spec(src)
        analyzer = LL1Analyzer(grammar)
        prepared, table = analyzer.build_table()
        write_generated_file(Path(args.output), prepared.start_symbol, table)
        print(f"OK: table in {args.output}")
    except (LexError, ParseError, GrammarError) as e:
        print(str(e))
        raise SystemExit(1)

if __name__ == "__main__":
    main()

```

Калькулятор

```

from __future__ import annotations

import argparse
import importlib.util
from pathlib import Path
from typing import List

from generator import EOF_SYMBOL, EPSILON, LL1Parser, \
    Node, Token, ParseError, LexError

class ExprLexer:
    def __init__(self, src: str):
        self.src = src
        self.i = 0
        self.line = 1
        self.col = 1

    def peek(self, k: int = 0) -> str:
        idx = self.i + k

```

```

        return self.src[idx] if idx < len(self.src) else "\0"

def advance(self) -> str:
    ch = self.peak()
    if ch == "\0":
        return ch
    self.i += 1
    if ch == "\n":
        self.line += 1
        self.col = 1
    else:
        self.col += 1
    return ch

def next_token(self) -> Token:
    while True:
        ch = self.peak()
        if ch == "\0":
            return Token(EOF_SYMBOL, "", self.line, self.col)
        if ch.isspace():
            self.advance()
            continue
        break

    line, col = self.line, self.col
    ch = self.peak()
    if ch in "+*()":
        self.advance()
        return Token(ch, ch, line, col)
    if ch.isdigit():
        buf = []
        while self.peak().isdigit():
            buf.append(self.advance())
        return Token("n", "".join(buf), line, col)
    raise LexError(f"Unexpected character '{ch}' at {line}:{col}")

def load_generated_table(path: str):
    module_path = Path(path)
    spec = importlib.util.spec_from_file_location("generated_table", module_path)
    if spec is None or spec.loader is None:
        raise RuntimeError(f"Cannot load table file: {path}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

```

```

def evaluate(node: Node) -> int:
    # E -> T E'
    # E' -> + T E' | ε
    # T -> F T'
    # T' -> * F T' | ε
    # F -> n | ( E )
    sym = node.symbol
    if sym == "E":
        return evaluate(node.children[0]) + evaluate_eprime(node.children[1])
    if sym == "T":
        return evaluate(node.children[0]) * evaluate_tprime(node.children[1])
    if sym == "F":
        if len(node.children) == 1 and node.children[0].symbol == "n":
            return int(node.children[0].token.lexeme)
        return evaluate(node.children[1])
    raise RuntimeError(f"Unexpected node symbol: {sym}")

def evaluate_eprime(node: Node) -> int:
    if not node.children or (len(node.children) == 1 \
        and node.children[0].symbol == EPSILON):
        return 0
    return evaluate(node.children[1]) + evaluate_eprime(node.children[2])

def evaluate_tprime(node: Node) -> int:
    if not node.children or (len(node.children) == 1 \
        and node.children[0].symbol == EPSILON):
        return 1
    return evaluate(node.children[1]) * evaluate_tprime(node.children[2])

def parse_expression(src: str, table_module) -> int:
    lexer = ExprLexer(src)
    tokens: List[Token] = []
    while True:
        tok = lexer.next_token()
        tokens.append(tok)
        if tok.type == EOF_SYMBOL:
            break

    parser = LL1Parser(table_module.SPEC_TABLE, table_module.START_SYMBOL)
    tree = parser.parse(tokens)
    return evaluate(tree)

```

```

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("table")
    ap.add_argument("input")
    args = ap.parse_args()

    table_module = load_generated_table(args.table)
    src = Path(args.input).read_text(encoding="utf-8")

    try:
        value = parse_expression(src, table_module)
        print(value)
    except (LexError, ParseError, Exception) as e:
        print(str(e))
        raise SystemExit(1)

if __name__ == "__main__":
    main()

```

Тестирование

Генератор компиляторов

Таблица для калькулятора

```

from __future__ import annotations

START_SYMBOL = 'E'
EOF_SYMBOL = '$'
EPSILON = ''
SPEC_TABLE = {
    'F': {'n': ['n'], '(': ['(', 'E', ')']},
    'T': {'(': ['F', "T"], 'n': ['F', "T"]},
    "T'": {'*': ['*', 'F', "T"], ')': [], '+': [], '$': []},
    "E'": {'+': ['+', 'T', "E"], ')': [], '$': []},
    'E': {'(': ['T', "E"], 'n': ['T', "E"]}
}

```

Таблица для собственной грамматики

```

from __future__ import annotations

START_SYMBOL = 'Program'
EOF_SYMBOL = '$'
EPSILON = ''

```

```

SPEC_TABLE = {
  'Alt': {'EPS': ['EPS'], 'STRING': ['Seq'], 'NONTERM': ['Seq']},
  'Alts': {
    'EPS': ['Alt', 'AltsTail'],
    'NONTERM': ['Alt', 'AltsTail'],
    'STRING': ['Alt', 'AltsTail']
  },
  'Decl': {'NONTERM': ['NONTERM'], 'LBRACE': ['LBRACE', 'NONTERM', 'RBRACE']},
  'Symbol': {'NONTERM': ['NONTERM'], 'STRING': ['STRING']},
  'AltsTail': {'COLON': ['COLON', 'Alt', 'AltsTail'], 'RANGLE': []},
  'SeqTail': {
    'STRING': ['Symbol', 'SeqTail'],
    'NONTERM': ['Symbol', 'SeqTail'],
    'RANGLE': [],
    'COLON': []
  },
  'Program': {'LBRACE': ['DeclList', 'Rules'], 'NONTERM': ['DeclList', 'Rules']},
  'DeclListTail': {'COMMA': ['COMMA', 'Decl', 'DeclListTail'], 'LANGLE': [], '$': []},
  'Seq': {'STRING': ['Symbol', 'SeqTail'], 'NONTERM': ['Symbol', 'SeqTail']},
  'DeclList': {'LBRACE': ['Decl', 'DeclListTail'], 'NONTERM': ['Decl', 'DeclListTail']},
  'Rule': {'LANGLE': ['LANGLE', 'NONTERM', 'COLON', 'Alts', 'RANGLE']},
  'Rules': {'LANGLE': ['Rule', 'Rules'], '$': []}
}

```

Калькулятор

Для тестового арифметического выражения:

$2+3*4*(0+9)$

Калькулятор вывел:

110

Вывод

Изучила алгоритм построения таблиц предсказывающего анализатора.